

# CPU Scheduling

## 3.1 INTRODUCTION

CPU scheduling is the procedure employed for **deciding to which of the ready processes, the CPU should be allocated**. CPU scheduling plays a pivotal role in the basic framework of the operating system owing to the fact that the **CPU is one of the primary resources of the computer system**.

The **algorithm used by the scheduler to carry out the selection of a process for execution** is known as **scheduling algorithm**.

A **number of scheduling algorithms** are available for CPU scheduling. **Each scheduling algorithm influences the resource utilization, overall system performance, and quality of service provided** to the user. Therefore, one has to reason out a number of criteria to be considered while selecting an algorithm on a particular system.

## 3.2 SCHEDULING CONCEPTS

### 4.2.1 When to Schedule

An important facet of scheduling is to determine **when the scheduler should make scheduling decisions**. The following circumstances may require the scheduler to make scheduling decisions.

- ***When a process switches from running to waiting state.*** This situation may occur in case the process has to wait for I/O or the termination of its child process or some another reason. In such situations, the scheduler has to select some ready process for execution.
- ***When a process switches from running to ready state due to occurrence of an interrupt.*** In such a situation, the scheduler may decide to run a process from the ready queue. If the interrupt was caused by some I/O device that has now completed its task, the scheduler may choose the process that was blocked waiting for the I/O.

- *When a process switches from waiting state to ready state.* This situation may occur when the process has completed its I/O operation. In such a situation, the scheduler may select either the process that has now come to the ready state or the current process may be continued.
- *When a process terminates and exits the system.* In this case, the scheduler has to select a process for execution from the set of ready processes.

### 4.2.2 Dispatcher

The **CPU scheduler** only selects a process to be executed next on the CPU but it cannot assign CPU to the selected process.

*The function of setting up the execution of the selected process on the CPU is performed by another module* of the operating system, known as **dispatcher**.

The dispatcher involves the following three steps to perform this function.

1. *Context switching is performed.* The kernel saves the context of currently running process and restores the saved state of the process selected by the CPU scheduler. In case the process selected by the short-term scheduler is new, the kernel loads its context.
2. *The system switches from the kernel mode to user mode* as a user process is to be executed.
3. *The execution of the user process selected by the CPU scheduler is started* by transferring the control either to the instruction that was supposed to be executed at the time the process was interrupted, or to the first instruction if the process is going to be executed for the first time after its creation.

**Note:** *The amount of time required by the dispatcher to suspend execution of one process and resume execution of another process is known as **dispatch latency**.*

*Low dispatch latency implies faster start of process execution.*

## 4.3 SCHEDULING CRITERIA

The scheduler must consider the following performance measures and optimization criteria in order to maximize the performance of the system.

- **Fairness:** It is defined as the degree to which each process is getting an equal chance to execute. The scheduler must ensure that each process should get a fair share of CPU time. However, it may treat different categories of processes (batch, real-time, or interactive) in a different manner.
- **CPU utilization:** It is defined as the percentage of time the CPU is busy in executing processes. For higher utilization, the CPU must be kept as busy as possible, that is, there must be some process running at all times.
- **Balanced utilization:** It is defined as the percentage of time all the system resources are busy. It considers not only the CPU utilization but the utilization of I/O devices, memory, and other resources also. To get more work done by the system, the CPU and I/O devices must be kept running simultaneously. For this, it is desirable to load a mixture of CPU-bound and I/O-bound processes in the memory.
- **Throughput:** It is defined as the total number of processes that a system can execute per unit of time. By and large, it depends on the average length of the processes to be executed. For the systems running long processes, throughput will be less as compared to the systems running short processes.
- **Turnaround time:** It is defined as the amount of time that has rolled by from the time of creation to the termination of a process. To put it differently, it is the difference between the time a process enters the system and the time it exits from the system. It includes all the time the process has spent waiting to enter into ready queue, within ready queue to get CPU, running on CPU, and in I/O queues. It is inversely proportional to the throughput, that is, more the turnaround time, less will be the throughput.

**Turnaround time = Exit time – Entry time**

- **Waiting time:** It is defined as the time used up by a process while

waiting in the ready queue. It does not take into account the execution time or time consumed for I/O. Thus, waiting time of a process can be determined as the difference between turnaround time and processing time. In practice, waiting time is a more accurate measure as compared to turnaround time.

**Waiting time= Turnaround Time – Burst Time**

- **Response time:** It is defined as the time elapsed after the user initiates a request and the system starts responding to this request.

For interactive systems, it is one of the best metric employed to gauge the performance. This is because in such systems, only the speed with which the system responds to the user's request matters and not the time it takes to output the response.

The basic purpose of a CPU scheduling algorithm is that it should tend to maximize fairness, CPU utilization, balanced utilization and throughput, and minimize turnaround, waiting and response time. Practically speaking, no scheduling algorithm optimizes all the scheduling criteria.

Thus, in general, the performance of an algorithm is evaluated on the basis of average measures. For example, an algorithm that minimizes the average waiting time is considered as a good algorithm because this improves the overall efficiency of the system. However, in the case of response time, minimizing the average is not a good criterion rather the variance in the response time of the processes should be minimized. This is because it is not desirable to have a process with long response time as compared to other processes.

## 4.4 SCHEDULING ALGORITHMS

A wide variety of algorithms are used for the CPU scheduling. These scheduling algorithms fall into two categories, namely, non-preemptive and preemptive.

- **Non-preemptive scheduling algorithms:** Once the CPU is allocated to a process, it cannot be taken back until the process voluntarily releases it (in case the process has to wait for I/O or some other event) or the process terminates.

In other words, we can say the decision to schedule a process is

made only when the currently running process either switches to the waiting state or terminates. In both cases, the CPU executes some other process from the set of ready processes.

Some examples of non-preemptive scheduling algorithms are first come first served (FCFS), shortest job first (SJF)

**Note:** A non-preemptive scheduling algorithm is also known as a **cooperative** or **voluntary** scheduling algorithm.

- **Preemptive scheduling algorithms:** The CPU can be forcibly taken back from the currently running process before its completion and allocated to some other process.

The **preempted process** is put back in the ready queue and resumes its execution when it is scheduled again. Thus, a process may be scheduled many times before its completion.

In preemptive scheduling, the decision to schedule another process is made whenever an interrupt occurs causing the currently running process to switch to ready state or a process having higher priority than the currently running process is ready to execute.

Some examples of preemptive scheduling algorithms are shortest remaining time next (SRTN), priority-based scheduling and round robin (RR) scheduling.

#### 4.4.1 First-Come First-Served (FCFS) Scheduling

FCFS is one of the simplest scheduling algorithms. As the name implies, the processes are executed in the order of their arrival in the ready queue, which means the process that enters the ready queue first gets the CPU first.

FCFS is a non-preemptive scheduling algorithm. Therefore, once a process gets the CPU, it retains the control of CPU until it blocks or terminates.

To implement FCFS scheduling, the implementation of ready queue is managed as a FIFO (First-in First-out) queue. When the first process enters the ready queue, it immediately gets the CPU and starts executing. Meanwhile, other processes enter the system and are added to the end of the queue by inserting their PCBs in the

queue.

When the currently running process completes or blocks, the CPU is allocated to the process at the front of the queue and its PCB is removed from the queue.

In case a currently running process was blocked and later it comes to the ready state, its PCB is linked to the end of queue.

**Example 1** Consider four processes P1, P2, P3, and P4 with their arrival times and required CPU burst (in milliseconds) as shown in the following table.

| Process        | P <sub>1</sub> | P <sub>2</sub> | P <sub>3</sub> | P <sub>4</sub> |
|----------------|----------------|----------------|----------------|----------------|
| Arrival time   | 0              | 2              | 3              | 5              |
| CPU burst (ms) | 15             | 6              | 7              | 5              |

How will these processes be scheduled according to FCFS scheduling algorithm? Compute the average waiting time and average turnaround time.

**Solution** The processes will be scheduled as depicted in the following **Gantt chart**.

|                |  |  |  |                |  |  |  |                |  |  |  |                |  |  |  |    |  |  |  |
|----------------|--|--|--|----------------|--|--|--|----------------|--|--|--|----------------|--|--|--|----|--|--|--|
| P <sub>1</sub> |  |  |  | P <sub>2</sub> |  |  |  | P <sub>3</sub> |  |  |  | P <sub>4</sub> |  |  |  |    |  |  |  |
| 0              |  |  |  | 15             |  |  |  | 21             |  |  |  | 28             |  |  |  | 33 |  |  |  |

Initially, P1 enters the ready queue at  $t = 0$  and CPU is allocated to it. While P1 is executing, P2, P3 and P4 enter the ready queue at  $t = 2$ ,  $t = 3$ , and  $t = 5$ , respectively. When P1 completes, CPU is allocated to P2 as it has entered before P3 and P4. When P2 completes, P3 gets the CPU after which P4 gets the CPU.

Since Turnaround time = Exit time – Entry time,

Turnaround time for P1=  $(15 - 0) = 15$  ms

Turnaround time for P2=  $(21 - 2) = 19$  ms

Turnaround time for P3=  $(28 - 3) = 25$  ms

Turnaround time for P4=  $(33 - 5) = 28$  ms

**Average turnaround time** =  $(15 + 19 + 25 + 28)/4 = 21.75$  ms

Since Waiting time = Turnaround time – Processing time,

Waiting time for  $P_1 = (15 - 15) = 0$  ms

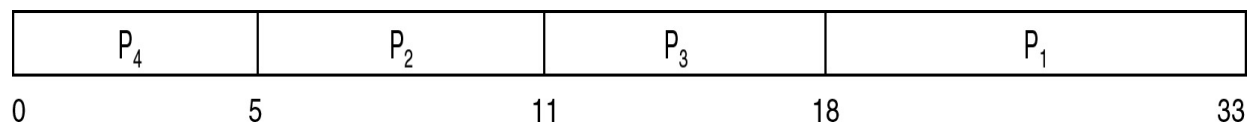
Waiting time for  $P_2 = (19 - 6) = 13$  ms

Waiting time for  $P_3 = (25 - 7) = 18$  ms

Waiting time for  $P_4 = (28 - 5) = 23$  ms

**Average waiting time** =  $(0 + 13 + 18 + 23)/4 = 13.5$  ms

The performance of FCFS scheduling algorithm largely depends on the order of arrival of processes in the ready queue, that is, whether the processes having long CPU burst enter before those having short CPU burst or vice versa. To illustrate this, assume that the processes (shown in Example 1) enter the ready queue in the order  $P_4$ ,  $P_2$ ,  $P_3$  and  $P_1$ . Now, the processes will be scheduled as shown in the following Gantt chart.



Using the above formulae, the average turnaround time and average waiting time can be computed as:

**Average turnaround time** =  $((5 - 0) + (11 - 2) + (18 - 3) + (33 - 5))/4 = 14.25$  ms

**Average waiting time** =  $((5 - 5) + (9 - 6) + (15 - 7) + (28 - 15))/4 = 6$  ms

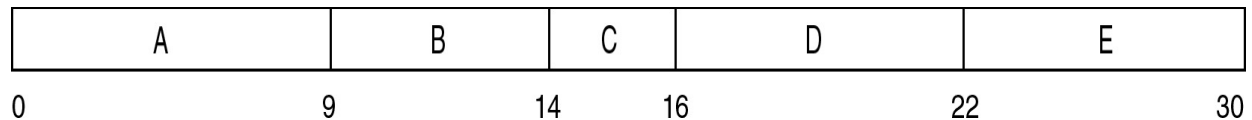
It is clear that if the processes having shorter CPU burst execute before those having longer CPU burst, the average waiting and turnaround time may reduce significantly.

**Example 2** Five jobs A through E arrive at a computer center with following details:

| <u>Job</u> | <u>Arrival time</u> | <u>CPU time (ms)</u> |
|------------|---------------------|----------------------|
| A          | 0                   | 9                    |
| B          | 1                   | 5                    |
| C          | 2                   | 2                    |
| D          | 3                   | 6                    |
| E          | 4                   | 8                    |

Calculate the turnaround time and waiting time for all processes applying FCFS algorithm.

**Solution** According to FCFS scheduling algorithm, the given processes will be scheduled as depicted in the following Gantt chart.



Since Turnaround time = Exit time – Entry time,

Turnaround time for A =  $(9 - 0) = 9$  ms

Turnaround time for B =  $(14 - 1) = 13$  ms

Turnaround time for c =  $(16 - 2) = 14$  ms

Turnaround time for D =  $(22 - 3) = 19$  ms

Turnaround time for E =  $(30 - 4) = 26$  ms

Since Waiting time = Turnaround time – Processing time,

Waiting time for A =  $(9 - 9) = 0$  ms

Waiting time for B =  $(13 - 5) = 8$  ms

Waiting time for c =  $(14 - 2) = 12$  ms

Waiting time for D =  $(19 - 6) = 13$  ms

Waiting time for E =  $(26 - 8) = 18$

ms

## Advantages

- It is **easier to understand and implement** as processes are simply to be added at the end and removed from the front of the queue. No process from in between the queue is required to be accessed.
- It is **well suited for batch systems** where the longer time periods for each process are often acceptable.

## Disadvantages

- The **average waiting time is not minimal**. Therefore, **this scheduling algorithm is never recommended where performance is a major issue**.
- It **reduces the CPU and I/O devices utilization under some circumstances**. For example, assume that there is one long CPU-bound process and many short I/O-bound processes in the ready queue. Now, it may happen that while the CPU-bound process is executing, the I/O-bound processes complete their I/O and come to the ready queue for execution. There they have to wait for the CPU-bound process to release the CPU and the I/O devices also remain idle during this time. When the CPU-bound process needs to perform I/O, it comes to the device queue and the CPU is allocated to I/O-bound processes. As the I/O-bound processes require a little CPU burst, they execute quickly and come back to the device queue thereby leaving the CPU idle. Then the CPU-bound process enters the ready queue and is allocated the CPU which again makes the I/O processes waiting in ready queue at some point of time. This happens again and again until the CPU-bound process is done, which results in low utilization of CPU and I/O devices.
- It is **not suitable for time sharing systems where each process should get the same amount of CPU time**.

### 4.4.2 Shortest Job First (SJF) Scheduling

The shortest job first, also known as **shortest process next (SPN)** or **shortest request next (SRN)**, is a **non-preemptive scheduling algorithm that schedules the processes according to the length of**

CPU burst they require.

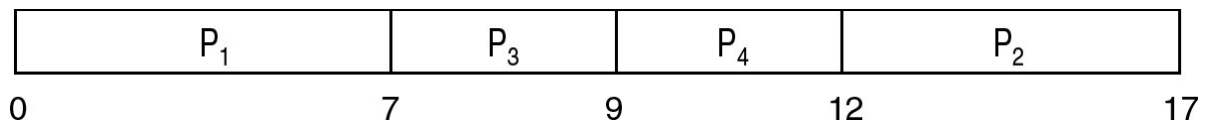
At any point of time, among all the ready processes, the one having the shortest CPU burst is scheduled first. Thus, a process has to wait until all the processes shorter than it have been executed. In case two processes have the same CPU burst, they are scheduled in the FCFS order.

**Example 3** Consider four processes P<sub>1</sub>, P<sub>2</sub>, P<sub>3</sub>, and P<sub>4</sub> with their arrival times and required CPU burst (in milliseconds) as shown in the following table.

| Process        | P <sub>1</sub> | P <sub>2</sub> | P <sub>3</sub> | P <sub>4</sub> |
|----------------|----------------|----------------|----------------|----------------|
| Arrival time   | 0              | 1              | 3              | 4              |
| CPU burst (ms) | 7              | 5              | 2              | 3              |

How will these processes be scheduled according to SJF scheduling algorithm? Compute the average waiting time and average turnaround time.

**Solution** The processes will be scheduled as depicted in the following Gantt chart.



Initially, P<sub>1</sub> enters the ready queue at t = 0 and gets the CPU as there are no other processes in the queue.

While it is executing, P<sub>2</sub>, P<sub>3</sub> and P<sub>4</sub> enter the queue at t = 1, t = 3 and t = 4, respectively.

When CPU becomes free, that is, at t = 7, it is allocated to P<sub>3</sub> because it has the shortest CPU burst among the three processes.

When P<sub>3</sub> completes, CPU is allocated first to P<sub>4</sub> and then to P<sub>2</sub>.

Since Turnaround time = Exit time – Entry time,

Turnaround time for P<sub>1</sub> = (7 – 0) = 7 ms

Turnaround time for P<sub>2</sub> = (17 – 1) = 16 ms

Turnaround time for P3=  $(9 - 3) = 6$  ms

Turnaround time for P4=  $(12 - 4) = 8$  ms

**Average turnaround time** =  $(7 + 16 + 6 + 8)/4 = 9.25$  ms

Since Waiting time = Turnaround time – Processing time,

Waiting time for P1=  $(7 - 7) = 0$  ms

Waiting time for P2=  $(16 - 5) = 11$  ms

Waiting time for P3=  $(6 - 2) = 4$  ms

Waiting time for P4=  $(8 - 3) = 5$  ms

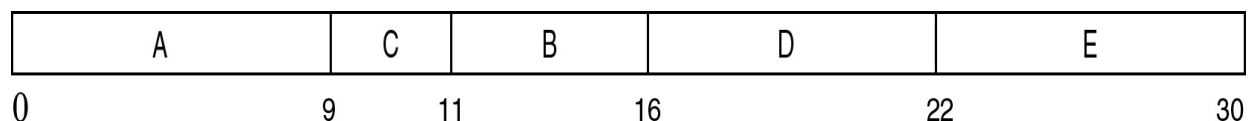
**Average waiting time** =  $(0 + 11 + 4 + 5)/4 = 5$  ms

**Example 4** Consider the same set of processes, their arrival times and CPU burst as shown.

| <u>Job</u> | <u>Arrival time</u> | <u>CPU time (ms)</u> |
|------------|---------------------|----------------------|
| A          | 0                   | 9                    |
| B          | 1                   | 5                    |
| C          | 2                   | 2                    |
| D          | 3                   | 6                    |
| E          | 4                   | 8                    |

How will these processes be scheduled according to SJF scheduling algorithm? Compute the average waiting time and average turnaround time.

**Solution** The processes will be scheduled as depicted in the following Gantt chart.



Since Turnaround time = Exit time – Entry time,

Turnaround time for A =  $(9 - 0) = 9$  ms

Turnaround time for B =  $(16 - 1) = 15$  ms

Turnaround time for C =  $(11 - 2) = 9$  ms

Turnaround time for D =  $(22 - 3) = 19$  ms

Turnaround time for E =  $(30 - 4) = 26$  ms

**Average turnaround time** =  $(9 + 15 + 9 + 19 + 26)/5 = 15.6$  ms

Since Waiting time = Turnaround time – Processing time,

Waiting time for A =  $(9 - 9) = 0$  ms

Waiting time for B =  $(15 - 5) = 10$  ms

Waiting time for C =  $(9 - 2) = 7$  ms

Waiting time for D =  $(19 - 6) = 13$  ms

Waiting time for E =  $(26 - 8) = 18$  ms

**Average waiting time** =  $(0 + 10 + 7 + 13 + 18)/5 = 9.6$  ms

## Advantages

- Short processes are made to run before the long ones which decreases the waiting time for short processes and increases the waiting time for long processes. However, the reduction in waiting time is more than the increment and thus, the average waiting time decreases.

## Disadvantages

- It is difficult to implement as it needs to know the length of CPU burst of processes in advance. In practice, having the prior knowledge of the required processing time of processes is difficult. Many systems expect users to provide estimates of CPU burst of processes which may not always be correct.
- It does not favour the processes having longer CPU burst. This is because as long as the short processes continue to enter the ready queue, the long processes will not be allowed to get the CPU. This results in starvation of long processes.

### 4.4.3 Shortest Remaining Time Next (SRTN) Scheduling

The shortest remaining time next also known as **shortest time to go (STG)** is a preemptive version of the SJF scheduling algorithm.

It takes into account the length of the remaining CPU burst of the processes rather than the whole length in order to schedule them. The scheduler **always chooses the process for execution that has the shortest remaining processing time.**

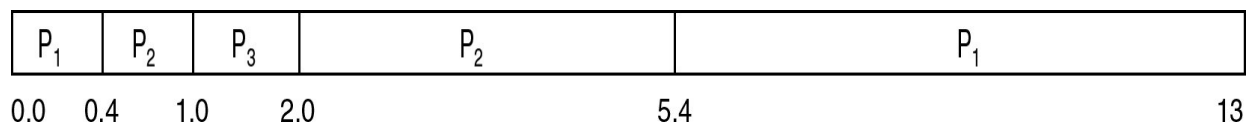
While a process is being executed, the CPU can be taken back from it and assigned to some newly arrived process if the CPU burst of the new process is shorter than its remaining CPU burst. Notice that if at any point of time, the remaining CPU burst of two processes becomes equal; they are scheduled in the FCFS order.

**Example 5** Suppose that the following processes arrive for the execution at the times indicated. Each process will run the listed amount of time.

| Process | Arrival time (ms) | Burst time (ms) |
|---------|-------------------|-----------------|
| $P_1$   | 0.0               | 8               |
| $P_2$   | 0.4               | 4               |
| $P_3$   | 1.0               | 1               |

What is the average turnaround time and average waiting time for these processes with SRTN algorithm?

**Solution** According to SRTN algorithm, the given processes will be scheduled as depicted in the following Gantt chart.



Initially,  $P_1$  enters the ready queue at  $t = 0.0$  and gets the CPU as there are no other processes in the queue. While it is executing, at time  $t = 0.4$ ,  $P_2$  with CPU burst of 4 ms enters the queue. At that time the remaining CPU burst of  $P_1$  is 7.6 ms which is greater than that of  $P_2$ . Therefore, the CPU is taken back from  $P_1$  and allocated to  $P_2$ . During execution of  $P_2$ ,  $P_3$  enters at  $t = 1.0$  with a CPU burst of 1 ms. Again CPU is switched from  $P_2$  to  $P_3$  as the remaining CPU burst of  $P_2$  at  $t = 1.0$  is 3.4 ms which is greater than that of  $P_3$ . When  $P_3$

completes at  $t = 2.0$ , the CPU is allocated to  $P_2$  because at that time the remaining CPU burst of  $P_2$  (which is, 3.4 ms) is shorter than that of  $P_1$  (which is 7.6 ms). Finally, when  $P_2$  completes its execution at  $t = 5.4$  ms, the CPU is allocated to  $P_1$  which completes its execution at  $t = 13$ .

Since Turnaround time = Exit time – Entry time,

Turnaround time for  $P_1 = (13 - 0) = 13$  ms

Turnaround time for  $P_2 = (5.4 - 0.4) = 5$  ms

Turnaround time for  $P_3 = (2 - 1) = 1$  ms

**Average turnaround time** =  $(13 + 5 + 1)/3 = 6.33$  ms

Since Waiting time = Turnaround time – Processing time,

Waiting time for  $P_1 = (13 - 8) = 5$  ms

Waiting time for  $P_2 = (5 - 4) = 1$  ms

Waiting time for  $P_3 = (1 - 1) = 0$  ms

**Average waiting time** =  $(5 + 1 + 0)/3 = 2$  ms

**Example 6** Consider the same set of processes, their arrival times and CPU burst as shown.

| Process        | $P_1$ | $P_2$ | $P_3$ | $P_4$ |
|----------------|-------|-------|-------|-------|
| Arrival time   | 0     | 1     | 3     | 4     |
| CPU burst (ms) | 7     | 5     | 2     | 3     |

How will these processes be scheduled according to SRTN scheduling algorithm? Compute the average waiting time and average turnaround time.

**Solution** The processes will be scheduled as depicted in the following Gantt chart.



Initially,  $P_1$  enters the ready queue at  $t = 0$  and gets the CPU as

there are no other processes in the queue. While it is executing, at time  $t = 1$ ,  $P_2$  with CPU burst of 5 ms enters the queue. At that time the remaining CPU burst of  $P_1$  is 6 ms which is greater than that of  $P_2$ . Therefore, the CPU is taken back from  $P_1$  and allocated to  $P_2$ . During execution of  $P_2$ ,  $P_3$  enters at  $t = 3$  with a CPU burst of 2 ms. Again CPU is switched from  $P_2$  to  $P_3$  as the remaining CPU burst of  $P_2$  at  $t = 3$  is 3 ms which is greater than that of  $P_3$ . However, when at time  $t = 4$ ,  $P_4$  with CPU burst of 3 ms enters the queue, the CPU is not assigned to it because at that time the remaining CPU burst of currently running process (that is,  $P_3$ ) is 1 ms which is shorter than that of  $P_4$ . When  $P_3$  completes, there are three processes  $P_1$ (6 ms),  $P_2$ (3 ms) and  $P_4$ (3 ms) in the queue. To break the tie between  $P_2$  and  $P_4$ , the scheduler takes into consideration their arrival order and the CPU is allocated first to  $P_2$ , then to  $P_4$  and finally, to  $P_1$ .

Since Turnaround time = Exit time – Entry time,

Turnaround time for  $P_1 = (17 - 0) = 17$  ms

Turnaround time for  $P_2 = (8 - 1) = 7$  ms

Turnaround time for  $P_3 = (5 - 3) = 2$  ms

Turnaround time for  $P_4 = (11 - 4) = 7$  ms

**Average turnaround time** =  $(17 + 7 + 2 + 7)/4 = 8.25$  ms

Since Waiting time = Turnaround time – Processing time,

Waiting time for  $P_1 = (17 - 7) = 10$  ms

Waiting time for  $P_2 = (7 - 5) = 2$  ms

Waiting time for  $P_3 = (2 - 2) = 0$  ms

Waiting time for  $P_4 = (7 - 3) = 4$  ms

**Average waiting time** =  $(10 + 2 + 0 + 4)/4 = 4$  ms

## Advantages

- A long process that is near to its completion may be favored over the short processes entering the system. This results in an improvement in the turnaround time of the long process.

## Disadvantages

- Like SJF, it also requires an estimate of the next CPU burst of a process in advance.
- Favoring a long process nearing its completion over the several short processes entering the system may affect the turnaround times of short processes.
- It favors only those long processes that are just about to complete and not those who have just started their operation. Thus, starvation of long processes still may occur.

### 4.4.4 Priority-based Scheduling

In priority-based scheduling algorithm, each process is assigned a priority and the higher priority processes are scheduled before the lower priority processes.

At any point of time, the process having the highest priority among all the ready processes is scheduled first.

In case two processes are having the same priority, they are executed in the FCFS order.

Priority scheduling may be either preemptive or non-preemptive.

The choice is made whenever a new process enters the ready queue while some process is executing. If the newly arrived process has the higher priority than the currently running process, the preemptive priority scheduling algorithm preempts the currently running process and allocates CPU to the new process. On the other hand, the non-preemptive scheduling algorithm allows the currently running process to complete its execution and the new process has to wait for the CPU.

**Note:** Both SJF and SRTN are special cases of priority-based scheduling where priority of a process is equal to inverse of the next CPU burst.

Lower is the CPU burst, higher will be the priority.

A major design issue related with priority scheduling is how to compute priorities of the processes.

The priority can be assigned to a process either internally defined

by the system depending on the process's characteristics like memory usage, I/O frequency, usage cost, etc., or externally defined by the user executing that process.

**Example 7** Consider four processes P<sub>1</sub>, P<sub>2</sub>, P<sub>3</sub>, and P<sub>4</sub> with their arrival times, required CPU burst (in milliseconds), and priorities as shown in the following table.

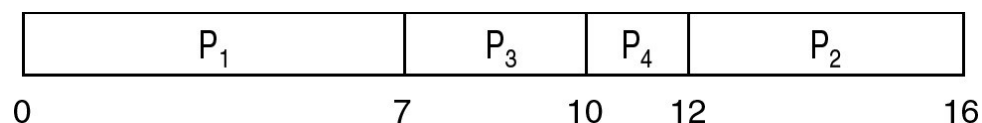
| Process        | P <sub>1</sub> | P <sub>2</sub> | P <sub>3</sub> | P <sub>4</sub> |
|----------------|----------------|----------------|----------------|----------------|
| Arrival time   | 0              | 1              | 3              | 4              |
| CPU burst (ms) | 7              | 4              | 3              | 2              |
| Priority       | 4              | 3              | 1              | 2              |

Assuming that the lower priority number means the higher priority, how will these processes be scheduled according to non-preemptive as well as preemptive priority scheduling algorithm? Compute the average waiting time and average turnaround time in both cases.

### **Solution**

#### **Non-preemptive priority scheduling algorithm**

The processes will be scheduled as depicted in the following Gantt chart.



Initially, P<sub>1</sub> enters the ready queue at t = 0 and gets the CPU as there are no other processes in the queue. While it is executing, P<sub>2</sub>, P<sub>3</sub>, and P<sub>4</sub> enter the queue at t = 1, t = 3, and t = 4, respectively. When CPU becomes free, that is, at t = 7, it is allocated to P<sub>3</sub> because it is having the highest priority (that is, 1) among the three processes. When P<sub>3</sub> completes, CPU is allocated to the next lower priority process, that is, P<sub>4</sub> and finally, the lowest priority process P<sub>2</sub> is executed.

Since Turnaround time = Exit time – Entry time,

Turnaround time for  $P_1 = (7 - 0) = 7$  ms

Turnaround time for  $P_2 = (16 - 1) = 15$  ms

Turnaround time for  $P_3 = (10 - 3) = 7$  ms

Turnaround time for  $P_4 = (12 - 4) = 8$  ms

**Average turnaround time** =  $(7 + 15 + 7 + 8)/4 = 9.25$  ms

Since Waiting time = Turnaround time – Processing time,

Waiting time for  $P_1 = (7 - 7) = 0$  ms

Waiting time for  $P_2 = (15 - 4) = 11$  ms

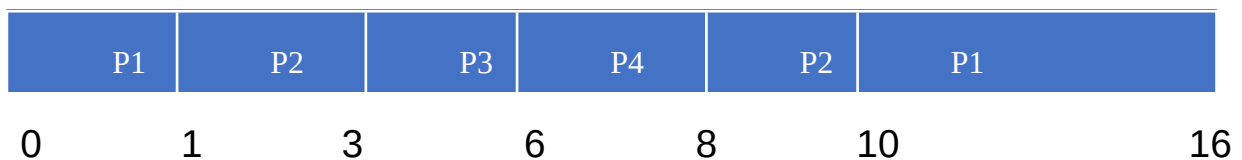
Waiting time for  $P_3 = (7 - 3) = 4$  ms

Waiting time for  $P_4 = (8 - 2) = 6$  ms

**Average waiting time** =  $(0 + 11 + 4 + 6)/4 = 5.25$  ms

### Preemptive priority scheduling algorithm

The processes will be scheduled as depicted in the following Gantt chart.



Initially,  $P_1$  of priority 4 enters the ready queue at  $t = 0$  and gets the CPU as there are no other processes in the queue. While it is executing, at time  $t = 1$ ,  $P_2$  of priority 3 greater than that of currently running process  $P_1$ , enters the queue. Therefore,  $P_1$  is preempted (with remaining CPU burst of 6 ms) and the CPU is allocated to  $P_2$ . During execution of  $P_2$ ,  $P_3$  of priority 1 enters at  $t = 3$ . Again CPU is switched from  $P_2$  (with remaining CPU burst of 2 ms) to  $P_3$  as the priority of  $P_3$  is greater than that of  $P_2$ . However, when at time  $t = 4$ ,  $P_4$  of priority 2 enters the queue, the CPU is not assigned to it because it has lower priority than currently running process  $P_3$ . When  $P_3$  completes, there are three processes  $P_1$ ,  $P_2$ , and  $P_4$  in the ready queue having priorities 4, 3, and 2, respectively. The CPU is allocated

first to  $P_4$ , then to  $P_2$  and finally to  $P_1$ .

Since Turnaround time = Exit time – Entry time,

Turnaround time for  $P_1 = (16 - 0) = 16$  ms

Turnaround time for  $P_2 = (10 - 1) = 9$  ms

Turnaround time for  $P_3 = (6 - 3) = 3$  ms

Turnaround time for  $P_4 = (8 - 4) = 4$  ms

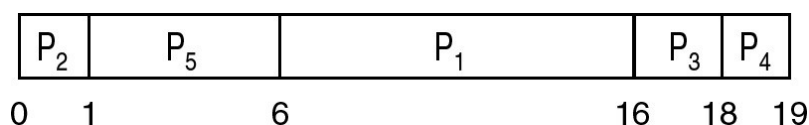
| Process | Burst time (ms) | Priority |
|---------|-----------------|----------|
| $P_1$   | 10              | 3        |
| $P_2$   | 1               | 1        |
| $P_3$   | 2               | 3        |
| $P_4$   | 1               | 4        |
| $P_5$   | 5               | 2        |

**Average waiting time** =  $(9 + 5 + 0 + 2)/4 = 4$  ms

**Example 8** Consider the following set of processes with the length of CPU burst time given in milliseconds.

Assume arrival order is:  $P_1, P_2, P_3, P_4, P_5$  all at time 0 and a smaller priority number implies a higher priority. Draw the Gantt charts illustrating the execution of these processes using preemptive priority scheduling.

**Solution** According to preemptive priority-based scheduling algorithm, the given processes will be scheduled as depicted in the following Gantt chart.



All processes enter the ready queue at  $t = 0$ . Since at that time,

the process  $P_2$  has the highest priority among all processes, the CPU is allocated to it. When  $P_2$  completes its execution at  $t = 1$ , the CPU is allocated to  $P_5$  as it has the highest priority among the remaining processes. Now at  $t = 6$ , a choice has to be made between processes  $P_1$  and  $P_3$  as both have the same priority. To break the tie between  $P_1$  and  $P_3$ , the scheduler takes into consideration their arrival order. Thus, the CPU is allocated first to  $P_1$  and then to  $P_3$ . Lastly at  $t = 18$ , the CPU is allocated to  $P_4$ .

## Advantages

- Important processes are never made to wait because of the execution of less important processes.

## Disadvantages

- It suffers from the problem of **starvation** of lower priority processes, since the continuous arrival of higher priority processes will prevent lower priority processes indefinitely from acquiring the CPU.
- One possible solution to this problem is **aging** - a process of gradually increasing the priority of a low priority process with increase in its waiting time. If the priority of a low priority process is increased after each fixed time interval, it is ensured that at some time it will become a highest priority process and get executed.

### 4.4.5 Round Robin (RR) Scheduling

The round robin scheduling is one of the most widely used preemptive scheduling algorithms which considers all the processes as equally important and treats them in a favorable manner.

Each process in the ready queue gets a fixed amount of CPU time (generally from 10 to 100 milliseconds) known as **time slice** or **time quantum** for its execution.

If the process does not execute completely till the end of time slice, it is preempted and the CPU is allocated to the next process in the ready queue.

However, if the process blocks or terminates before the time slice expires, the CPU is switched to the next process in the ready queue at that moment only.

To implement the round robin scheduling algorithm, the ready queue is treated as a circular queue.

All the processes arriving in the ready queue are put at the end of queue.

The CPU is allocated to the first process in the queue, and the process executes until its time slice expires. If the CPU burst of the process being executed is less than one time quantum, the process itself releases the CPU and is deleted from the queue. The CPU is then allocated to the next process in the queue.

However, if the process does not execute completely within the time slice, an interrupt occurs when the time slice expires. The currently running process is preempted, put back at the end of the queue and the CPU is allocated to the next process in the queue.

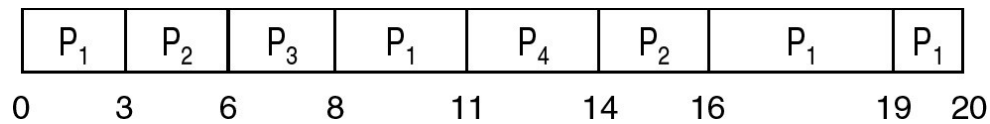
The preempted process again gets the CPU after all the processes before it in the queue have been allocated their CPU time slice. The whole process continues until all the processes in queue have been

| Process        | P <sub>1</sub> | P <sub>2</sub> | P <sub>3</sub> | P <sub>4</sub> |
|----------------|----------------|----------------|----------------|----------------|
| Arrival time   | 0              | 1              | 3              | 4              |
| CPU burst (ms) | 10             | 5              | 2              | 3              |

arrival times and required CPU burst (in milliseconds) as shown in the following table.

Assuming that the time slice is 3 ms, how will these processes be scheduled according to round robin scheduling algorithm? Compute the average waiting time and average turnaround time.

**Solution** The processes will be scheduled as depicted in the following Gantt chart.



Initially,  $P_1$  enters the ready queue at  $t = 0$  and gets the CPU for 3 ms. While it executes,  $P_2$  and  $P_3$  enter the queue at  $t = 1$  and  $t = 3$ , respectively. Since,  $P_1$  does not execute within 3 ms, an interrupt occurs when the time slice gets over.  $P_1$  is preempted (with remaining CPU burst of 7 ms), put back in the queue after  $P_3$  because  $P_4$  has not entered yet and the CPU is allocated to  $P_2$ . During execution of  $P_2$ ,  $P_4$  enters in the queue at  $t = 4$  and put at the end of queue after  $P_1$ . When  $P_2$  times out, it is preempted (with remaining CPU burst of 2 ms) and put back at the end of queue after  $P_4$ . The CPU is allocated to the next process in the queue, that is, to  $P_3$  and it executes completely before the time slice expires. Thus, the CPU is allocated to the next process in the queue which is  $P_1$ .  $P_1$  again executes for 3 ms, then preempted (with remaining CPU burst of 4 ms) and put back at the end of the queue after  $P_2$  and the CPU is allocated to  $P_4$ .  $P_4$  executes completely within the time slice and the CPU is allocated to next process in the queue, that is,  $P_2$ . As  $P_2$  completes before the time out occurs, the CPU is switched to  $P_1$  at  $t = 16$  for another 3 ms. When the time slice expires, CPU is again allocated to  $P_1$  as it is the only process in the queue.

Since Turnaround time = Exit time – Entry time,

Turnaround time for  $P_1 = (20 - 0) = 20$  ms

Turnaround time for  $P_2 = (16 - 1) = 15$  ms

Turnaround time for  $P_3 = (8 - 3) = 5$  ms

Turnaround time for  $P_4 = (14 - 4) = 10$  ms

**Average turnaround time** =  $(20 + 15 + 5 + 10)/4 = 12.5$  ms

Since Waiting time = Turnaround time – Processing time,

Waiting time for  $P_1 = (20 - 10) = 10$  ms

Waiting time for  $P_2 = (15 - 5) = 10$  ms

Waiting time for  $P_3 = (5 - 2) = 3$  ms

Waiting time for  $P_4 = (10 - 3) = 7$  ms

**Average waiting time** =  $(10 + 10 + 3 + 7)/4 = 7.5$  ms

**Example 12** Five jobs A through E arrive at a computer center with following details:

| <u>Job</u> | <u>Arrival Time</u> | <u>CPU Time (ms)</u> |
|------------|---------------------|----------------------|
| A          | 0                   | 9                    |
| B          | 1                   | 5                    |
| C          | 2                   | 2                    |
| D          | 3                   | 6                    |
| E          | 4                   | 8                    |

Calculate the average turnaround time and average waiting time of the processes applying Round Robin algorithm.

**Solution** According to this scheduling algorithm with quanta = 3, the given processes will be scheduled as depicted in the following Gantt chart.

|   |   |   |    |    |    |    |    |    |    |    |   |   |
|---|---|---|----|----|----|----|----|----|----|----|---|---|
|   | A | B | C  | D  | A  | E  | B  | D  | A  | E  | E | 0 |
| 3 | 6 | 8 | 11 | 14 | 17 | 19 | 22 | 25 | 28 | 30 |   |   |

Since Turnaround time = Exit time – Entry time,

Turnaround time for A =  $(25 - 0) = 25$  ms

Turnaround time for B =  $(19 - 1) = 18$  ms

Turnaround time for C =  $(8 - 2) = 6$  ms

Turnaround time for D =  $(22 - 3) = 19$  ms

Turnaround time for E =  $(30 - 4) = 26$  ms

**Average turnaround time** =  $(25 + 18 + 6 + 19 + 26)/5 = 18.8$  ms

Since Waiting time = Turnaround time – Processing time,

Waiting time for A =  $(25 - 9) = 16$  ms

Waiting time for B =  $(18 - 5) = 13$  ms

Waiting time for C =  $(6 - 2) = 4$  ms

Waiting time for D =  $(19 - 6) = 13$  ms

Waiting time for E =  $(26 - 8) = 18$  ms

**Average waiting time** =  $(16 + 13 + 4 + 13 + 18)/5 = 12.8$  ms

Note that the performance of round robin scheduling is greatly affected by the size of the time quantum.

If the time quantum is too small, a number of context switches occur which in turn increase the system overhead. The more time will be spent in performing context switching rather than executing the processes.

On the other hand, if the time quantum is too large, the performance of round robin simply degrades to FCFS.

**Note:** If the time quantum is too small, say  $1\ \mu\text{s}$ , the round robin scheduling is called **processor sharing**.

## Advantages

- It is efficient for time sharing systems where the CPU time is divided among the competing processes.
- It increases the fairness among the processes.

## Disadvantages

- The processes (even the short processes) may take long time to execute. This decreases the system throughput.
- It requires some extra hardware support such as a timer to cause interrupt after each time out.

**Note:** Ideally, the size of time quantum should be such that 80% of the processes could complete their execution within one time quantum.

### 4.4.6 Multilevel Queue Scheduling

The multilevel queue scheduling is designed for the environments where the processes can be categorized into different groups on the

basis of their different response time requirements or different scheduling needs.

One possible categorization may be based on whether the process is a system process, batch process, or an interactive process (see Figure 4.2).

Each group of processes is associated with a specific priority.

For example, the system processes may have the highest priority whereas the batch processes may have the least priority.

To implement multilevel scheduling algorithm, the ready queue is partitioned into as many separate queues as there are groups.

Whenever a new process enters, it is assigned permanently to one of the ready queues depending on its properties like memory requirements, type and priority.

Each ready queue has its own scheduling algorithm.

For example, for batch processes, FCFS scheduling algorithm may be used, and for interactive processes, one may use the round robin scheduling algorithm.

In addition, the processes in higher priority queues are executed before those in lower priority queues. This implies no batch process can run unless all the system processes and interactive processes have been executed completely.

Moreover, if a process enters into a higher priority queue while a process in lower priority queue is executing, then the lower priority process would be preempted in order to allocate the CPU to the higher priority process.

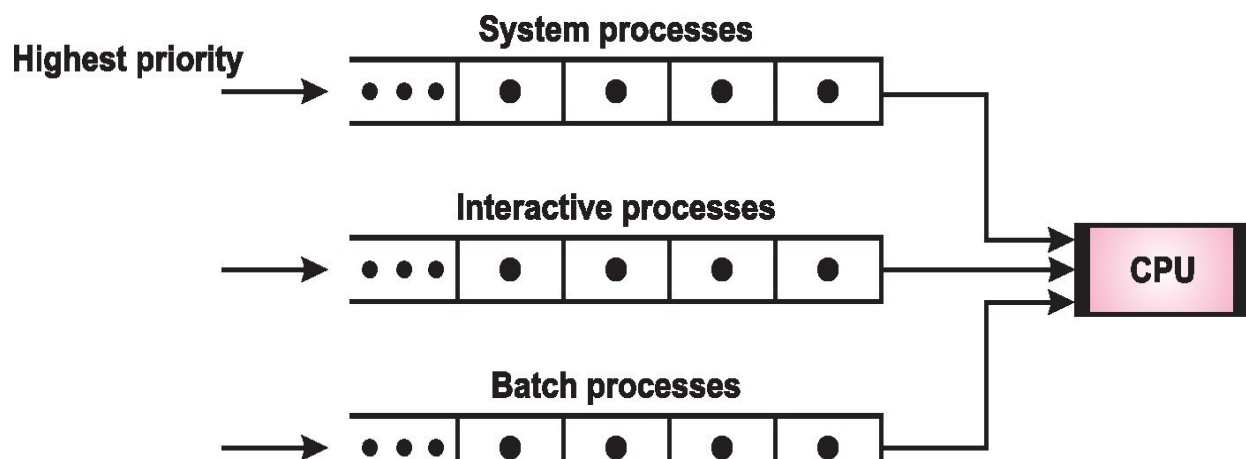


Fig. 4.2 Multilevel Queue Scheduling

## Advantages

- Processes are permanently assigned to their respective queues and do not move between queues. This results in low scheduling overhead.

## Disadvantages

- The processes in lower priority queues may have to **starve** for CPU in case processes are continuously arriving in higher priority queues.

**Solution:** One possible way to prevent starvation is to **time slice among the queues**. Each queue gets a certain share of CPU time which it schedules among the processes in it. Note that the **time slice of different priority queues may differ**.

### 4.4.7 Multilevel Feedback Queue Scheduling

The multilevel feedback queue scheduling also known as **multilevel adaptive scheduling** is an improved version of multilevel queue scheduling algorithm.

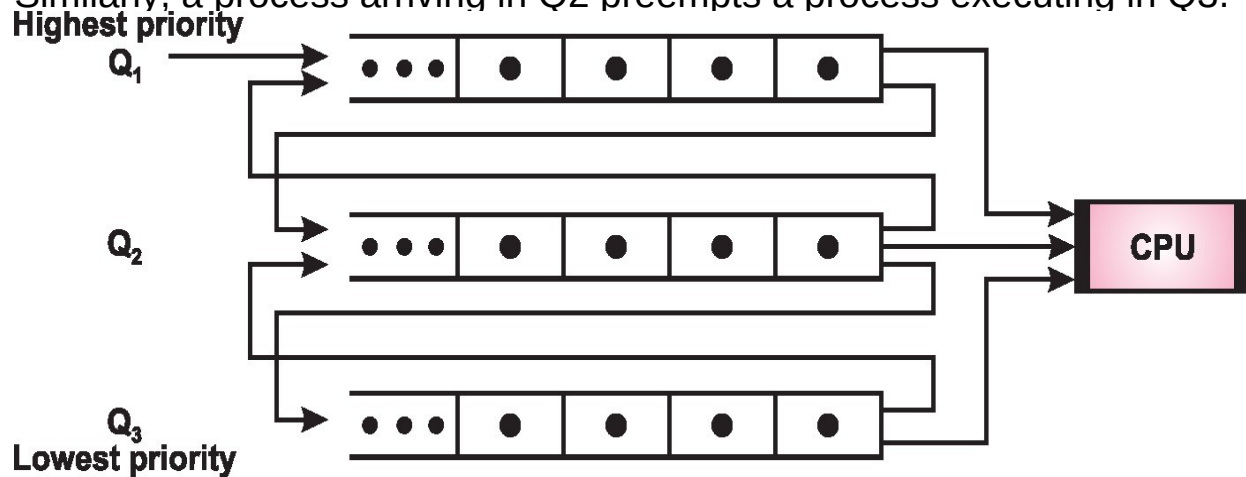
In this scheduling algorithm, **processes are not permanently assigned to queues; instead they are allowed to move between the queues**.

The decision to **move a process between queues is based on the time taken by it in execution so far and its waiting time**.

If a process uses too much CPU time, it is moved to a lower priority queue. Similarly, a process that has been waiting for too long in a lower priority queue is moved to a higher priority queue in order to avoid starvation.

To understand this algorithm, consider a multilevel feedback queue scheduler (see Figure 4.3) with three queues, namely, Q1, Q2, and Q3. Further, **assume that the queues Q1 and Q2 employ round robin scheduling algorithm with time quantum of 5 ms and 10 ms,**

respectively while in queue Q3, the processes are scheduled in FCFS order. The scheduler first executes all processes in Q1. When Q1 is empty, the scheduler executes the processes in Q2. Finally, when both Q1 and Q2 are empty, the processes in Q3 are executed. While executing processes in Q2, if a new process arrives in Q1, the currently executing process is preempted and the new process starts executing. Similarly, a process arriving in Q2 preempts a process executing in Q3.



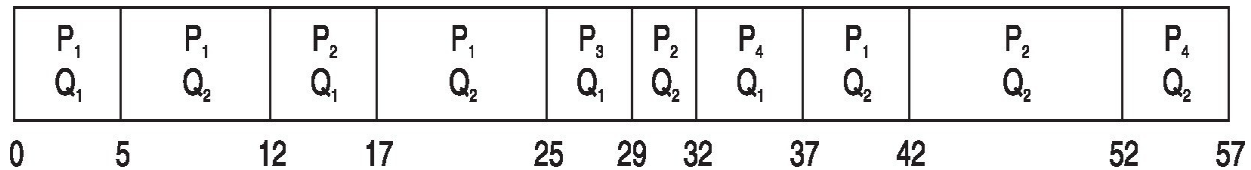
**Fig. 4.3** Multilevel Feedback Queue Scheduling

**Example 13** Consider four processes  $P_1$ ,  $P_2$ ,  $P_3$ , and  $P_4$  with their arrival times and required CPU burst (in milliseconds) as shown in the following table.

Assume that there are three ready queues  $q_1$ ,  $q_2$  and  $q_3$ . The CPU time slice for  $q_1$  and  $q_2$  is 5 ms and 10 ms, respectively and in  $q_3$ , processes are scheduled on FCFS basis. How will these processes be scheduled according to multilevel feedback queue scheduling algorithm? Compute the average waiting time and average turnaround time.

| Process      | $P_1$ | $P_2$ | $P_3$ | $P_4$ |
|--------------|-------|-------|-------|-------|
| Arrival time | 0     | 12    | 25    | 32    |

**Solution** The processes will be scheduled as depicted in the following Gantt chart.



Initially, P<sub>1</sub> enters the system at t = 0, placed in q<sub>1</sub> and allocated the CPU for 5 ms. Since, it does not execute completely, it is moved to q<sub>2</sub> at t = 5. Now q<sub>1</sub> is empty so the scheduler picks up the process from the head of q<sub>2</sub>. Since, P<sub>1</sub> is the only process in q<sub>2</sub>, it is again allocated the CPU for 10 ms. But during its execution, P<sub>2</sub> enters q<sub>1</sub> at t = 12, therefore P<sub>1</sub> is preempted and P<sub>2</sub> starts executing. At t = 17, P<sub>2</sub> is moved to q<sub>2</sub> and placed after P<sub>1</sub>. The CPU is allocated to the first process in q<sub>2</sub>, that is, P<sub>1</sub>. While P<sub>1</sub> is executing, P<sub>3</sub> enters q<sub>1</sub> at t = 25 so P<sub>1</sub> is preempted, placed after P<sub>2</sub> in q<sub>2</sub> and P<sub>3</sub> starts executing. As P<sub>3</sub> executes completely within time slice, the scheduler picks up the first process in q<sub>2</sub> which is P<sub>2</sub> at t = 29. While P<sub>2</sub> is executing, P<sub>4</sub> enters q<sub>1</sub> at t = 32 because of which P<sub>2</sub> is preempted and placed after P<sub>1</sub> in q<sub>2</sub>. The CPU is assigned to P<sub>4</sub> for 5 ms and at t = 37, P<sub>4</sub> is moved to q<sub>2</sub> and placed after P<sub>2</sub>. At the same time, the CPU is allocated to P<sub>1</sub>(first process in q<sub>2</sub>). When it completes at t = 42, the next process in q<sub>2</sub> which is P<sub>2</sub>, starts executing. When it completes, the last process in q<sub>2</sub>, that is, P<sub>4</sub> is executed.

Since Turnaround time = Exit time – Entry time, we have:

Turnaround time for P<sub>1</sub> = (42 – 0) = 42 ms

Turnaround time for P<sub>2</sub> = (52 – 12) = 40 ms

Turnaround time for P<sub>3</sub> = (29 – 25) = 4 ms

Turnaround time for P<sub>4</sub> = (57 – 32) = 25 ms

**Average turnaround time** =  $(42 + 40 + 4 + 25)/4 = 27.75$  ms

Since Waiting time = Turnaround time – Processing time, we have:

Waiting time for  $P_1 = (42 - 25) = 17$

Waiting time for  $P_2 = (40 - 18) = 22$  ms

Waiting time for  $P_3 = (4 - 4) = 0$  ms

Waiting time for  $P_4 = (25 - 10) = 15$  ms

**Average waiting time** =  $(17 + 22 + 0 + 15)/4 = 13.5$  ms

## Advantages

- It is fair to I/O-bound (short) processes as these processes need not wait too long and are executed quickly.
- It **prevents starvation** by moving a lower priority process to a higher priority queue if it has been waiting for too long.

## Disadvantages

- It is the most complex scheduling algorithm.
- Moving the processes between queues causes a number of context switches which results in an increased overhead.
- The turnaround time for long processes may increase significantly.